Multi-Wavelength Object Generator

User Manual and Technical Documentation

Stuart Jefferies

November 11, 2025

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Key Features . . . 3

2 Theoretical Background **3**
2.1 Wavelength Sampling . . . 3
2.2 Material Reflectance Models . . . 4
2.2.1 Material Types . . . 4
2.3 Gaussian Absorption Features . . . 4
2.4 Binary Star Modeling . . . 5

3 Input Parameters **5**
3.1 Physical Parameters . . . 5
3.2 Telescope Parameters . . . 5
3.3 Wavelength Parameters . . . 6
3.4 Simulation Parameters . . . 6
3.5 Absorption Band Parameters . . . 6

4 Template File Format **7**
4.1 Template Structure . . . 7
4.2 Creating Template Files . . . 7

5 Output Files **8**
5.1 Primary Output: multi_wavelength_object.fits . . . 8
5.2 Data Values . . . 8
5.3 Visualization Output . . . 8

6 Running the Code **8**
6.1 Prerequisites . . . 8
6.2 Basic Execution . . . 9
6.3 Typical Workflow . . . 9

7 Integration with Broadband_AI.py **10**
7.1 Parameter Compatibility . . . 10
7.2 Data Flow . . . 10

8 Advanced Usage **10**
8.1 Custom Material Types . . . 10
8.2 Multiple Absorption Bands . . . 11
8.3 Wavelength Resolution Studies . . . 11

9 Troubleshooting **11**
9.1 Common Issues . . . 11
9.2 Validation . . . 12

10 Performance Considerations **12**
10.1 Computational Complexity . . . 12
10.2 Memory Requirements . . . 12
10.3 Optimization Tips . . . 13

11 Mathematical Appendix **13**
11.1 Polynomial Coefficients . . . 13
11.1.1 Material 1: Solar Panel . . . 13
11.1.2 Material 2: Kapton . . . 13
11.2 Gaussian Absorption Derivation . . . 13

12 References and Resources **13**
12.1 Related Codes . . . 13
12.2 Key Algorithms . . . 14
12.3 Physical Constants . . . 14

13 Version History **14**

14 Acknowledgments **14**

1 Introduction

The `multiwavelength_object_generator.py` code generates three-dimensional hyperspectral object cubes for use in astronomical adaptive optics simulations. The code supports both satellite material modeling with wavelength-dependent reflectance properties and binary star systems with blackbody radiation spectra.

1.1 Purpose

This code generates realistic multi-wavelength objects that can be convolved with point spread functions (PSFs) in atmospheric turbulence simulations. The output is specifically designed to work with the `Broadband_AI.py` simulation code, which performs adaptive optics compensation and hyperspectral deconvolution.

1.2 Key Features

- Seven distinct satellite material types with physically-based reflectance spectra
- Novel material type with Gaussian absorption features for spectral signature studies
- Support for Closely Spaced Objects (CSO) with scaled reflectance
- Binary star blackbody radiation modeling
- Wavelength sampling matched to Fourier-domain adaptive optics simulations
- FITS file output with proper coordinate conventions
- Visualization tools for material spectral signatures

2 Theoretical Background

2.1 Wavelength Sampling

The code uses a specific wavelength sampling scheme designed to match the Fourier-domain representation used in adaptive optics simulations. For a given wavelength λ, the spatial frequency extent in the Fourier plane is:

$$r(\lambda) = \frac{\lambda_{\min} \cdot \dim}{4\lambda} \tag{1}$$

where $\lambda_{\min}$ is the minimum wavelength and dim is the dimension of the phase screen.
The wavelength slices are computed as:

$$r_{\max} = \frac{\dim}{4} \tag{2}$$

$$r_{\min} = \left\lfloor r_{\max} \cdot \frac{\lambda_{\min}}{\lambda_{\max}} \right\rfloor + 1 \tag{3}$$

$$n_{\text{slice}} = \left\lfloor \frac{r_{\max} - r_{\min}}{\Delta_{\text{slice}}} \right\rfloor + 1 \tag{4}$$

$$\lambda_k = \frac{\lambda_{\min} \cdot \dim}{4 \cdot r_k} \tag{5}$$

where Δ_{slice} is the slice increment (controlled by the `incr` parameter) and $r_k = r_{\min} + k \cdot \Delta_{\text{slice}}$.

2.2 Material Reflectance Models

Each material type is modeled using a polynomial reflectance function:

$$R(\lambda) = \sum_{n=0}^{N} c_n \lambda^n \tag{6}$$

where c_n are empirically-derived coefficients for each material type and λ is wavelength in nanometers.

2.2.1 Material Types

Table 1: Satellite material types and template values

ID	Material	Description
1	Solar Panel	Standard photovoltaic material
2	Kapton	Polyimide film insulation
3	Aluminized Mylar	Reflective thermal blanket
4	Kapton (Aged)	UV-degraded polyimide
5	Aluminized Mylar (Aged)	Degraded thermal blanket
6	CSO Solar Panel	Scaled solar panel for unresolved objects
7	Solar Panel with Absorption	Solar panel + Gaussian absorption lines

Note: Material 6 uses the same spectral shape as Material 1 but scaled by the `CSO_scale` factor (default: 0.01) to simulate Closely Spaced Objects that appear dimmer.

2.3 Gaussian Absorption Features

Material 7 introduces narrow-band absorption features superimposed on the solar panel spectrum. The absorption is modeled as:

$$T(\lambda) = 1 - (1 - T_{\min}) \exp\left[-\frac{(\lambda - \lambda_c)^2}{2\sigma^2}\right] \tag{7}$$

where:

- $T(\lambda)$ is the transmission factor
- $T_{\min}$ is the minimum transmission (reflectivity) at band center
- λ_c is the center wavelength of the absorption band
- $\sigma = \text{FWHM}/(2\sqrt{2\ln 2})$ is the Gaussian width

The modified reflectance is then:

$$R_{\text{modified}}(\lambda) = R_{\text{base}}(\lambda) \prod_i T_i(\lambda) \tag{8}$$

where the product is over all absorption bands.

2.4 Binary Star Modeling

For binary star systems (`obj_type = 0`), the spectral radiance follows the Planck blackbody function:

$$B(\lambda, T) = \frac{8\pi hc}{\lambda^5} \frac{1}{\exp(hc/\lambda k_B T) - 1} \tag{9}$$

where:

- $h = 6.6260693 \times 10^{-34}$ J·s is Planck's constant
- $c = 2.997 \times 10^8$ m/s is the speed of light
- $k_B = 1.380658 \times 10^{-23}$ J/K is Boltzmann's constant
- T is the temperature in Kelvin

3 Input Parameters

All input parameters are set in the code header and must be edited before running the simulation.

3.1 Physical Parameters

Table 2: Physical and atmospheric parameters (for binary stars only)

Parameter	**Variable**	**Default**	**Units**
Star 1 Temperature	`T1`	6000	K
Star 2 Temperature	`T2`	4000	K
Air Pressure	`P`	600	mm Hg
Air Temperature	`T`	7	°C
Water Vapor Pressure	`fwv`	8	mm Hg
Zenith Angle	`zen`	0	degrees

3.2 Telescope Parameters

Table 3: Telescope configuration parameters

Parameter	**Variable**	**Default**	**Units**
Aperture Diameter	`D`	360.0	cm
Phase Screen Diameter	`D1`	720.0	cm

The phase screen diameter is set to $D_1 = 2D$ to ensure adequate sampling of atmospheric turbulence.

3.3 Wavelength Parameters

Table 4: Wavelength range specification

Parameter	Variable	Default	Units
Minimum Wavelength	`lamb1`	400.0	nm
Maximum Wavelength	`lamb2`	1000.0	nm

The wavelength range should span the visible to near-infrared for most satellite tracking applications.

3.4 Simulation Parameters

Table 5: Core simulation parameters

Parameter	Variable	Default	Description
Wavelength Increment	`incr`	1	Fourier slice spacing
Fried Parameter at 500nm	`r0_500`	17	cm
Number of Images	`nim`	1	(legacy, not used)
Object Type	`obj_type`	1	0=binary, else satellite
CSO Scale Factor	`CSO_scale`	0.01	Material 6 scaling
Display Plots	`display`	1	0=off, 1=on
Template Filename	`template_filename`	"template.fits"	Path to template

Critical Parameter: The `incr` parameter controls spectral resolution. Smaller values produce more wavelength slices but increase computation time. This parameter **must match** the value used in `Broadband_AI.py`.

3.5 Absorption Band Parameters

Material 7 (solar panel with absorption lines) requires specification of absorption bands:

```
absorption_bands = [
    (center_nm, fwhm_nm, min_reflectivity),
    (center_nm, fwhm_nm, min_reflectivity),
    ...
]
```

Table 6: Absorption band parameter specification

Parameter	Units	Description
`center_nm`	nm	Center wavelength of absorption feature
`fwhm_nm`	nm	Full width at half maximum
`min_reflectivity`	fraction	Minimum reflectivity at band center

Default Configuration:

```
absorption_bands = [
    (500, 35, 0.05),    # Band 1: 5% reflectivity at 500 nm
    (900, 70, 0.15)     # Band 2: 15% reflectivity at 900 nm
]
```

This creates two absorption features that can be used to test hyperspectral deconvolution algorithms.

4 Template File Format

For satellite objects (`obj_type` $\neq 0$), a template FITS file defines the spatial distribution of materials.

4.1 Template Structure

The template is a 2D FITS image where each pixel value indicates the material type:

- **Value 0:** Empty space (no object)
- **Value 1:** Solar panel
- **Value 2:** Kapton insulation
- **Value 3:** Aluminized mylar
- **Value 4:** Aged kapton
- **Value 5:** Aged aluminized mylar
- **Value 6:** CSO solar panel (scaled)
- **Value 7:** Solar panel with absorption lines

4.2 Creating Template Files

Templates can be created using any FITS-compatible software. Key requirements:

1. Dimensions must match the computed `dim` value
2. Values must be integers in the range [0, 7]
3. Coordinate system: origin at bottom-left (FITS convention)

Example using Python:

```
import numpy as np
from astropy.io import fits

# Create a template (example: 512x512)
template = np.zeros((512, 512), dtype=int)

# Define object geometry
template[200:300, 200:300] = 1  # Solar panel region
```

```
template[250:280, 250:280] = 7  # Central absorption feature

# Save as FITS
fits.writeto('template.fits', template, overwrite=True)
```

5 Output Files

5.1 Primary Output: multi_wavelength_object.fits

The main output is a 3D FITS cube containing the hyperspectral object:

- **Format:** FITS binary table
- **Dimensions:** (nslice, dim, dim)
- **Axis 0:** Wavelength slices (shortest to longest)
- **Axis 1:** Spatial dimension (x-axis)
- **Axis 2:** Spatial dimension (y-axis)
- **Coordinate Convention:** FITS standard (origin at bottom-left)

5.2 Data Values

The object cube contains material reflectance values scaled by 0.01:

$$\text{object}[x, y, \lambda] = 0.01 \times R(\lambda) \times T(\lambda) \tag{10}$$

where $R(\lambda)$ is the base reflectance and $T(\lambda)$ is the absorption transmission (1.0 for materials without absorption).

5.3 Visualization Output

When `display = 1`, the code generates a plot showing spectral signatures for each material type present in the template. This plot displays:

- Wavelength (nm) on x-axis
- Object value (reflectance) on y-axis
- Separate trace for each material type
- Legend indicating material ID and pixel location

6 Running the Code

6.1 Prerequisites

Required Python packages:

```
numpy
matplotlib
astropy
aotools
```

Installation:

```
pip install numpy matplotlib astropy aotools
```

6.2 Basic Execution

1. Edit input parameters in the code header
2. Ensure template file exists (if using satellite objects)
3. Run the code:

```
python3 multiwavelength_object_generator.py
```

4. Output file `multi_wavelength_object.fits` will be created

6.3 Typical Workflow

Step 1: Define Object Geometry

- Create or modify template FITS file
- Set material types in template pixels
- Save template with appropriate filename

Step 2: Configure Parameters

- Set `template_filename` to your template path
- Adjust wavelength range (`lamb1`, `lamb2`)
- Set spectral resolution via `incr`
- Configure absorption bands if using material 7

Step 3: Generate Object

- Run the code
- Verify output dimensions and wavelength sampling
- Examine spectral plots (if `display = 1`)

Step 4: Use in Simulation

- Use generated `multi_wavelength_object.fits` as input to `Broadband_AI.py`
- Ensure matching `incr`, `D`, `lamb1`, `lamb2` parameters

7 Integration with Broadband_AI.py

The output from `multiwavelength_object_generator.py` is designed to be loaded directly by `Broadband_AI.py`.

7.1 Parameter Compatibility

The following parameters **must match** between the two codes:

Table 7: Required matching parameters

Parameter	Reason
`incr`	Determines wavelength sampling
`lamb1`	Minimum wavelength
`lamb2`	Maximum wavelength
`D`	Telescope diameter affects dimensions

7.2 Data Flow

1. `multiwavelength_object_generator.py` creates object cube
2. Object saved as FITS file with proper coordinate system
3. `Broadband_AI.py` loads object using `load_multiwavelength_object()`
4. Coordinate transformations applied automatically
5. Object convolved with atmospheric PSFs
6. Deconvolution recovers object from noisy data

8 Advanced Usage

8.1 Custom Material Types

To add new material types beyond the existing seven:

1. Add polynomial coefficients to the `coefficients` dictionary
2. Extend the `conditions` and `choices` lists
3. Update template file to use new material ID

Example:

```
# Add material 8 coefficients
coefficients[8] = np.array([a0, a1, a2, ...])

# Extend conditions and choices
conditions = [template == i for i in range(1, 9)]
choices = [
    values[1], values[2], values[3],
```

```
    values[4], values[5],
    CSO_scale * values[1],
    values[7],
    values[8]  # New material
]
```

8.2 Multiple Absorption Bands

The absorption band system supports arbitrary numbers of features:

```
# Complex absorption spectrum
absorption_bands = [
    (450, 20, 0.30),   # Weak blue absorption
    (550, 30, 0.10),   # Strong green absorption
    (650, 25, 0.40),   # Medium red absorption
    (850, 50, 0.20),   # Broad NIR absorption
]
```

Each band is applied multiplicatively, allowing complex spectral signatures.

8.3 Wavelength Resolution Studies

The `incr` parameter can be varied to study spectral resolution effects:

Table 8: Spectral resolution vs computational cost

incr	Typical nslice	Relative Cost
1	200–300	4×
2	100–150	2×
4	50–75	1× (baseline)
8	25–38	0.5×

Recommendation: Use `incr = 1` for final production runs, larger values for initial testing.

9 Troubleshooting

9.1 Common Issues

Problem: "No pixels found for material 7"

Solution: Either the template doesn't contain value 7, or the template file wasn't loaded correctly. Check:

- `template_filename` path is correct
- Template contains pixels with value 7
- `obj_type` is set to 1 (satellite mode)

Problem: Dimension mismatch with `Broadband_AI.py`
Solution: Verify that `D`, `lamb1`, `lamb2`, and `incr` match exactly between codes.
Problem: Absorption features not visible
Solution: Check that:

- Absorption band centers fall within [`lamb1`, `lamb2`]
- FWHM is large enough relative to wavelength spacing
- `min_reflectivity` is sufficiently small (e.g., < 0.5)
- Template contains material 7 pixels

9.2 Validation

To verify correct operation:

1. Run with `display = 1`
2. Check that nslice matches expected value
3. Examine wavelength sampling printout
4. Verify spectral plot shows expected material signatures
5. For material 7, confirm absorption dips at specified wavelengths

10 Performance Considerations

10.1 Computational Complexity

The primary computational cost scales as:

$$\text{Time} \propto \text{nslice} \times \text{dim}^2 \tag{11}$$

Typical execution times on modern hardware:

- `dim` = 512, `incr` = 4: ∼0.1–0.5 seconds
- `dim` = 1024, `incr` = 2: ∼1–2 seconds
- `dim` = 2048, `incr` = 1: ∼5–10 seconds

10.2 Memory Requirements

Peak memory usage:

$$\text{Memory} \approx 8 \times \text{dim}^2 \times \text{nslice bytes} \tag{12}$$

For typical parameters:

- `dim` = 512, `nslice` = 200: ∼400 MB
- `dim` = 1024, `nslice` = 100: ∼800 MB
- `dim` = 2048, `nslice` = 50: ∼1.6 GB

10.3 Optimization Tips

- Use `incr` = 4 for development, 1 for production
- Vectorized operations already implemented for efficiency
- Loading large templates may benefit from memory-mapped FITS files
- Display plots can be disabled (`display = 0`) for batch processing

11 Mathematical Appendix

11.1 Polynomial Coefficients

Material reflectance polynomials are evaluated as:

$$R(\lambda) = c_0 + c_1\lambda + c_2\lambda^2 + \cdots + c_n\lambda^n \tag{13}$$

11.1.1 Material 1: Solar Panel

$$R_1(\lambda) = 966.7504 - 4.5836\lambda + 8.008 \times 10^{-3}\lambda^2 - 6.111 \times 10^{-6}\lambda^3 + 1.739 \times 10^{-9}\lambda^4 \tag{14}$$

11.1.2 Material 2: Kapton

$$\begin{aligned} R_2(\lambda) = &- 1.196 \times 10^5 + 1.460 \times 10^3\lambda - 7.648\lambda^2 + 2.247 \times 10^{-2}\lambda^3 \\ &- 4.056 \times 10^{-5}\lambda^4 + 4.616 \times 10^{-8}\lambda^5 - 3.239 \times 10^{-11}\lambda^6 \\ &+ 1.283 \times 10^{-14}\lambda^7 - 2.200 \times 10^{-18}\lambda^8 \end{aligned} \tag{15}$$

11.2 Gaussian Absorption Derivation

Starting from the normalized Gaussian:

$$G(\lambda) = \exp\left[-\frac{(\lambda - \lambda_c)^2}{2\sigma^2}\right] \tag{16}$$

The transmission function is constructed to satisfy boundary conditions:

$$T(\lambda_c) = T_{\min} \quad \text{(at band center)} \tag{17}$$

$$T(\lambda \to \infty) = 1 \quad \text{(far from band)} \tag{18}$$

This gives:

$$T(\lambda) = 1 - (1 - T_{\min})G(\lambda) \tag{19}$$

The FWHM is related to σ by:

$$\text{FWHM} = 2\sqrt{2\ln 2}\,\sigma \approx 2.355\sigma \tag{20}$$

12 References and Resources

12.1 Related Codes

- `Broadband_AI.py`: Atmospheric turbulence simulation and deconvolution
- `solar_panel.py`: Original absorption band development code

12.2 Key Algorithms

- Wavelength sampling: Matches Fourier representation in AO codes
- Material modeling: Empirical polynomial fits to laboratory measurements
- Absorption features: Gaussian profiles for spectral line simulation
- Blackbody radiation: Planck function for stellar objects

12.3 Physical Constants

All physical constants are defined using CODATA recommended values:

$$c = 2.997 \times 10^{8} \text{ m/s} \tag{21}$$

$$h = 6.6260693 \times 10^{-34} \text{ J·s} \tag{22}$$

$$k_B = 1.380658 \times 10^{-23} \text{ J/K} \tag{23}$$

13 Version History

Table 9: Code development history

Version	Date	Changes
1.0	—	Initial implementation with 5 materials
2.0	—	Added CSO support (material 6)
3.0	November 11, 2025	Added absorption bands (material 7)

14 Acknowledgments

This code builds upon earlier work in atmospheric turbulence simulation and adaptive optics. The material reflectance models are based on empirical measurements of spacecraft materials.